ECTE331 Project
Part1: Fault-Tolerant Autonomous Drone Navigation System in Java

Reference: Lecture 1 and Java related lecture notes.

Aim:
Design and implement a Java application that simulates a fault-tolerant autonomous drone navigation system. The system demonstrates: Fault-tolerance, Redundancy, Majority Voting, Reliability Monitoring, Exception Handling, and simple File Logging.

System Overview:
The drone estimates its altitude using three redundant sensors: Sensor A, Sensor B, and Sensor C. Each sensor produces an **integer** simulated altitude reading. A controller decides the final altitude using majority voting (Triple Modular Redundancy: TMR).

Functional Requirements:

1. Sensor Reading Generation
Each sensor must generate an **integer** reading using the method `readSensor(…)`, where valid altitude value is in the range [0:200] meters. The value can be generated using a similar code : baselineValue+ `random.nextInt`(Range)[1]

2. Fault Simulation Rules
Each sensor reading must simulate one of the following outcomes:

Random Range	Behaviour
0–14	Sensor failure (exception thrown)
15–29	Corrupted reading
30–99	Valid reading

Pseudo code:
Define a variable *chance* to hold a generated random value between 0 and 99 inclusive.
`If` *chance*< 15 , then throw a `SensorReadException` object
`Else if` *chance*< 30 , then generate a random value outside of the valid range [0:200]
`Else` generate a valid sensor reading value

3. Corrupted Data Rule
Corrupted values are invalid altitude readings out of the valid range [0:200] meters

4. Custom Exceptions
Your application should make use of the two following custom Exception classes:

- `SensorReadException`: Represents sensor failure
- `SystemReliabilityException`: Thrown when system has to enter the SAFE MODE.

[1] This method generate a random integer value between 0 and "Range-1"

The code of the two exception classes is given below

```
import java.io.IOException;
public class SensorReadException extends IOException
{
public SensorReadException(String message)
{
super(message);
}
}  //class

public class SystemReliabilityException extends Exception
{

public SystemReliabilityException(String message)
{
super(message);
}
} //class
```

5. Majority Voting Rule

The final altitude is determined as per the following rules:

- if two valid sensors outputs are equal, then use that value
- if ALL sensor outputs differ, fallback to the previous valid drone altitude value

6. Reliability Rule

A reliability failure occurs when:

- fewer than 2 valid sensor readings exist (i.e. at least two sensors fail), OR
- no majority is found among the sensor reading

If two consecutive failures occur, the system must:

i) throw `SystemReliabilityException`
ii) enter SAFE MODE,
iii) stop execution,

Hint: You can use in the `main()` method a `try-catch` block, with a loop inside the try block to process the sensor readings according to the above specifications; in the case of two consecutive failures, the sensor processing code should generate a `SystemReliabilityException` exception which is then caught by the catch block to log the transition to SAFE mode and terminate the application execution.

7. Logging Requirement

All important events, see below, must be logged to a file named "log.txt". For simplicity, no backup file is required. Each log entry should be prefixed with the date and time of the event and should include the IDs of the outlier sensor(s).

- **Required Logging Events**
  - sensor failure

- corrupted reading
- outlier detection
- majority decision
- fallback decision
- SAFE MODE activation

Console Output: Must display
- sensor values
- corrupted/failure events
- voting decision
- reliability status
- SAFE MODE if triggered

What to submit:
- Detailed solution documentation generated using `JavaDoc`
- Code concisely commented according to `JavaDoc` utility requirements
- Log file
- A report, with reference to the log file, demonstrating all possible use cases of the application.

Note: you might have to run the application multiple times, with different range value of `randomInt` to reproduce all the possible application use cases. The log file name does not need to be the same for every test; you can generate a new file name dynamically for each application run (a random value suffix appended to “log”).